Hybrid Feature-Aware Matrix Factorization for Cold-Start Skincare Retrieval

A from-scratch WARP-loss implementation (the LightFM algorithm)
with a dedicated cold-start evaluation protocol

Patrick Oliveira

October 15, 2026

Abstract

We implement hybrid, feature-aware matrix factorization trained with a Weighted Approximate-Rank Pairwise (WARP) loss – the algorithm popularized by LightFM – entirely from scratch in NumPy/SciPy, for top-N product retrieval on the Sephora Products and Skincare Reviews dataset. Unlike the two identity-indexed sibling projects in this portfolio (BPR over anime titles, SVD over MovieLens ratings), user and item representations here are sums of *feature* embeddings – skin type and tone for users; brand, category, price tier and ingredient tags for products – rather than sums indexed by raw identity. This is the classical, pre-deep-learning answer to cold start: a user or item the model has never seen individually can still be scored, as long as it shares known features with the training population. We report a dedicated cold-start evaluation slice, built by withholding a population of users and items from training entirely, and show the hybrid model retrieves measurably better-than-popularity candidates for them, while an ID-only matrix-factorization baseline and a pure popularity baseline both collapse to a score of exactly zero on the same slice – not a close loss, but a structural inability to say anything at all.

Contents

1 Notation **3**

2 Introduction **3**
2.1 The hybrid alternative . . . 3
2.2 Why not depend on the `lightfm` package . . . 3
2.3 Scope and claims . . . 3
2.4 Repository . . . 4

3 Data **4**
3.1 Synthetic fixture . . . 4
3.2 Implicit feedback . . . 5

4 Feature Engineering **5**
4.1 Declarative specification . . . 5
4.2 Point-in-time correctness . . . 5
4.3 A deliberate exclusion from the retrieval representation . . . 5
4.4 Testing surface . . . 6

5 Methodology **6**
5.1 Score function . . . 6
5.2 WARP loss . . . 7
5.3 Gradients and the update rule . . . 7
5.4 Baselines . . . 8

6 Evaluation Protocol **8**
6.1 Warm-start: leave-last-one-out . . . 8
6.2 Cold-start: entities withheld entirely . . . 8
6.3 Metrics . . . 9
6.4 Hyperparameter selection . . . 9

7 Results **9**
7.1 Warm-start . . . 9
7.2 Cold-start . . . 10
7.3 Honest caveats . . . 10

8 Conclusion **10**
8.1 What transfers . . . 10
8.2 Future work . . . 11

1 Notation

Let $\mathcal{U}$ be the set of users and $\mathcal{I}$ the set of items. Each user u has a set of known features $F_u \subseteq \mathcal{F}_{\text{user}}$ (skin type, skin tone, and – only for users seen during training – an identity token); each item i has a set of known features $F_i \subseteq \mathcal{F}_{\text{item}}$ (brand, category, price bucket, ingredient tags, and – only for items seen during training – an identity token). Every feature f owns a learned embedding $\mathbf{e}_f \in \mathbb{R}^d$. Implicit feedback is the set of observed positive interactions $\mathcal{R} \subseteq \mathcal{U} \times \mathcal{I}$, and $\mathcal{I}_u^+ = \{\, i \in \mathcal{I} : (u, i) \in \mathcal{R} \,\}$ denotes the products a user reviewed positively.

2 Introduction

Every recommender in this portfolio is implicit-feedback matrix factorization; what differs between them is what gets factorized. The anime project factorizes a 73k × 12k matrix of raw user and item identities with a BPR loss [2]; the MovieLens project factorizes an explicit rating matrix with truncated SVD. Both share the same structural limitation: a row or column exists only for an identity observed during training. A user who signs up the day after the last retraining job, or a product added to the catalogue this morning, has no row or column at all, and the only fallback available to an identity-indexed model is non-personalized popularity.

2.1 The hybrid alternative

LightFM [1] reframes the factorization: instead of learning one embedding per user and one per item, it learns one embedding per *feature*, and represents a user or item as the sum of the embeddings of its known features,

$$\mathbf{p}_u = \sum_{f \in F_u} \mathbf{e}_f, \qquad \mathbf{q}_i = \sum_{g \in F_i} \mathbf{e}_g. \tag{1}$$

A brand-new user who has filled in a skin-type and skin-tone profile already has a well-defined $\mathbf{p}_u$, built from embeddings the model learned from every *other* user who shares those two attributes. The same holds for a brand-new product described by its brand, category, price tier and ingredient tags. Identity is folded in as just one more feature – present for users/items observed during training, deliberately absent otherwise – so the model degrades gracefully to a pure content-based recommender rather than failing outright.

This is a classical, linear, SGD-trained model: there are no hidden layers and no nonlinear activations between the feature sum and the score. The contribution of this report is not a new loss function; it is (a) a from-scratch, dependency-free implementation of the WARP loss that LightFM popularized, and (b) a cold-start evaluation protocol precise enough to quantify the gap between a hybrid and an identity-only model, rather than asserting it qualitatively.

2.2 Why not depend on the `lightfm` package

The `lightfm` PyPI package exists and implements the same idea, but its Cython extension is compiled against a fixed, old NumPy ABI and routinely fails to build on current Python/NumPy combinations – exactly the kind of fragile dependency a portfolio repository should not ask a reviewer to fight with. Reimplementing the algorithm in plain NumPy/SciPy (Section 5) removes that dependency entirely while keeping every equation in this report directly traceable to a function in the repository.

2.3 Scope and claims

1. The feature layer is point-in-time correct. Aggregate features are computed with a strict `<` cutoff against each user's own interaction order, never `<=`, and this is enforced by tests that inject a future review and assert no feature value changes (Section 4).

2. Cold-start is an operationalized test, not a claim. A population of users and items is withheld from training *entirely* – not merely their last interaction – and the identity-feature vocabulary is built only from the training partition, so a cold entity provably has no ID token available (Section 6).

3. The comparison is fair. The ID-only baseline is the same family of pairwise SGD factorization (Section 5), not a strawman; it loses on the cold-start slice for a structural reason (no row/column exists for a withheld entity), not a tuning one.

2.4 Repository

The implementation, infrastructure, and the scripts that generate every table in this document are available in the accompanying repository. The results tables (Tables 4 and 5) are emitted by `scripts/export_results_table.py` and the feature catalogue (Table 2) by `scripts/export_feature_tables.py`, so the document cannot drift away from the code.

3 Data

The target dataset is the Sephora Products and Skincare Reviews collection (Kaggle ref. `nadyinky/sephora-pro` roughly 8,000 products with attributes such as category, brand, price and ingredient lists, and on the order of one million reviews carrying a rating, a recommendation flag, and reviewer attributes including skin type and skin tone.

3.1 Synthetic fixture

The repository ships a synthetic generator (`scripts/download_data.py`) that reproduces this schema without requiring Kaggle credentials, so the test suite and CI run with zero external dependencies. Products are assigned a brand, a category, a price drawn from a category-appropriate range, and two to four ingredient tags; one tag is deliberately drawn from a fixed list of tags each skin type responds well to (Table 1), so the catalogue always contains genuine structure for every skin type to recover, not just noise.

Table 1: Synthetic ingredient-affinity clusters. A real dataset would not have this structure made explicit; the synthetic fixture encodes it so that a model with the right inductive bias (shared ingredient features) can be shown to recover it with a bounded sample size.

Skin type	Preferred ingredient tags
Dry	Hyaluronic Acid, Ceramides, Squalane, Glycerin
Oily	Salicylic Acid, Niacinamide, Clay, Zinc PCA
Combination	Niacinamide, Hyaluronic Acid, Green Tea
Normal	Vitamin C, Peptides, Antioxidant Complex
Sensitive	Centella Asiatica, Oat Extract, Ceramides, Panthenol

Each synthetic user is assigned a skin type and a skin tone, and reviews products with a selection probability weighted by the overlap between the product's ingredient tags and the user's skin-type cluster, mildly tilted by a long-tailed popularity prior. Ratings are drawn around a centre that is higher for well-matched products and lower otherwise, and a review counts as "recommended" only when the rating clears the implicit-feedback threshold (Section 6). Every selection decision is keyed off *features*, never off the raw user or product ID – this is what gives a feature-aware model real structure to recover that an ID-only model cannot, by construction, generalize from one identity to another.

3.2 Implicit feedback

A review is treated as a positive interaction when rating ≥ 4 or the reviewer explicitly marked the product as recommended; everything else is dropped rather than treated as an explicit negative, following the usual implicit-feedback convention. An iterative k-core filter then removes users and products with fewer than five surviving positives, looping because removing a product can push a user below the floor and vice versa.

4 Feature Engineering

In the sibling BPR and SVD projects, the feature layer produces auxiliary signal: confidence weights, popularity priors, content features consulted only as a cold-start fallback. Here the feature layer *is* the model's representation – there is no separate identity-indexed embedding table that the features merely augment. This changes what correctness means for the pipeline: a bug in a feature transform is not a secondary signal degrading gracefully, it is a bug in the thing being factorized.

4.1 Declarative specification

`conf/features.yaml` declares every feature once, consumed identically by training and by the serving path (`tests/test_features_parity.py` asserts the two never drift apart). Each declaration carries a *fit scope*:

- **catalogue** – known the moment a product is listed (brand, category, price bucket, ingredient tags). Always available, cold item or not.
- **profile** – a static, self-reported user attribute (skin type, skin tone). Not a statistic, so not subject to the train-only rule; a brand-new user supplies it at signup.
- **train_only** – a vocabulary or statistic restricted to the training partition. Both identity features (`user_identity`, `item_identity`) use this scope: the vocabulary of ID tokens literally does not contain an entry for a withheld entity, which is what makes cold-start an operationalized property of the pipeline rather than an assertion about it.

Table 2 lists every declared feature with its family, kind and scope; the per-feature rationale (Table 3) documents why each one was declared.

4.2 Point-in-time correctness

Aggregate features derived from the interaction log (e.g. `user_review_count_bucket`) accept a strict cutoff and filter with `<`, never `<=`, against each user's own `interaction_order`. `tests/test_features_lea` operationalizes this directly: it injects a review positioned after the cutoff for an existing user and asserts that no feature value for that user changes, and separately asserts that a review positioned *exactly at* the cutoff is excluded (the boundary is exclusive, not inclusive). Item-level aggregates (`item_popularity_bucket`) are instead protected by a `fit_on: train_only` contract, verified by asserting the statistic computed on the training partition differs from the same statistic computed on the full dataset – if they matched, the held-out interactions would already be baked into a value the model is scored on.

4.3 A deliberate exclusion from the retrieval representation

`item_popularity_bucket` and `item_catalogue_context` are declared, registered and leakage-tested like every other feature, but are not included in the item representation the hybrid model actually factorizes. The reason is structural rather than a matter of taste: a withheld

Table 2: Declared features. "Scope" is the fit boundary: catalogue and profile features are known a priori; train_only features – including both identity features – are restricted to the training partition, which is what makes cold users/items genuinely lack an ID token rather than merely an unseen one.

Feature	Family	Kind	Scope	PIT
`user_skin_type`	user	categorical	profile	–
`user_skin_tone`	user	categorical	profile	–
`user_identity`	user	identity	train_only	–
`item_brand`	item	categorical	catalogue	–
`item_category`	item	categorical	catalogue	–
`item_price_bucket`	item	categorical	catalogue	–
`item_ingredient_tags`	item	multi_categorical	catalogue	–
`item_identity`	item	identity	train_only	–
`user_review_count_bucket`	interaction	categorical	train_only	✓
`item_popularity_bucket`	interaction	categorical	train_only	–
`item_catalogue_context`	context	categorical	train_only	–

(cold) item has, by construction, exactly zero training interactions, so both aggregates collapse every cold item onto the same "unpopular" / "new_arrival" token. Including them in $\mathbf{q}_i$ would teach the model to systematically suppress cold items through a learned global popularity prior at precisely the moment we want to test whether it can still retrieve them through shared brand/category/ingredient structure. The ID-only and popularity baselines already encode an item-popularity prior through their own bias terms – a prior that is, for the same structural reason, unavailable for a cold item – so nothing is lost from the comparison by leaving these two aggregates out of the hybrid model's own representation; they remain fully part of the feature store, the schema and leakage tests, and the feature table above.

4.4 Testing surface

Three test files operationalize the three properties above: `test_features_schema.py` validates `conf/features.yaml` against the registry (every declared transform is implemented and vice versa, every identity feature is declared `train_only`, every feature documents a rationale); `test_features_leakage.py` implements the point-in-time and fit-scope checks described above, plus a label-shuffling sanity check that destroys the user–item association and asserts ranking quality collapses to the random floor; `test_features_parity.py` asserts that `recompute_user_tokens/recomp` – the functions the serving path calls for an entity outside the last training run – reproduce exactly the tokens the batch-built feature store used during training, for every warm user/item sampled.

5 Methodology

5.1 Score function

Both the user and item representations are sums of feature embeddings (Equation 1, repeated here for the score derivation):

$$\mathbf{p}_u = \sum_{f \in F_u} \mathbf{e}_f, \qquad \mathbf{q}_i = \sum_{g \in F_i} \mathbf{e}_g. \tag{2}$$

The score of a user–item pair is their inner product plus an item bias,

$$s(u,i) = \Big\langle \sum_{f \in F_u} \mathbf{p}_f,\ \sum_{g \in F_i} \mathbf{q}_g \Big\rangle = \langle \mathbf{p}_u, \mathbf{q}_i \rangle + b_i, \tag{3}$$

where $b_i = \sum_{g \in F_i} \beta_g$ is itself a sum of per-feature bias scalars, mirroring the embedding sum. A per-user bias is deliberately omitted: it would shift every item's score for that user by the same constant and therefore cancel out of the pairwise margin used by the loss below, exactly as the per-user bias does in the sibling BPR project's own derivation – it would carry zero gradient under this objective.

5.2 WARP loss

Weighted Approximate-Rank Pairwise loss [3, 1] optimizes a pairwise ranking criterion like BPR, but instead of weighting every violated pair equally, it weights a positive pair by an *estimate of the positive's current rank* in the full catalogue, so that correcting a badly mis-ranked positive contributes far more gradient than nudging one that is already well-placed.

For a positive pair (u, i), negatives j are sampled uniformly and rejected until a margin violation is found,

$$s(u,j) > s(u,i) - 1, \tag{4}$$

or until $N_{\max}$ trials are exhausted. Let N be the number of trials taken. Because negatives are sampled uniformly at random, the probability that a given negative violates the margin is approximately $\mathrm{rank}(i)/(|\mathcal{I}| - 1)$, so the expected number of trials before the first violation is the reciprocal of that probability; inverting gives the rank estimate

$$\mathrm{rank}(i) \approx \left\lfloor \frac{|\mathcal{I}| - 1}{N} \right\rfloor. \tag{5}$$

Few trials imply many items already violate the margin, i.e. i is ranked low; many trials imply i is already hard to beat, i.e. ranked high. The hinge loss for the triplet is weighted by the margin-ranking loss function

$$L(\mathrm{rank}(i)) = \sum_{r=1}^{\mathrm{rank}(i)} \frac{1}{r}, \tag{6}$$

a harmonic sum that grows (slowly) with rank, so that

$$\ell(u,i,j) = L(\mathrm{rank}(i)) \cdot \max\big(0,\ 1 - s(u,i) + s(u,j)\big). \tag{7}$$

5.3 Gradients and the update rule

Write $m = 1 - s(u,i) + s(u,j)$ for the margin and $w = L(\mathrm{rank}(i))$ for the WARP weight; $\ell = w \cdot \max(0, m)$ is non-zero, and has a gradient, only when $m > 0$. For an active triplet,

$$\frac{\partial \ell}{\partial s(u,i)} = -w, \qquad \frac{\partial \ell}{\partial s(u,j)} = +w. \tag{8}$$

Since $s(u,i) = \langle \mathbf{p}_u, \mathbf{q}_i \rangle + b_i$ and every feature embedding is shared across every entity that carries that feature, the chain rule distributes the gradient to *every active feature embedding* of u, i and j simultaneously:

$$\mathbf{p}_u\text{'s features } f: \ \mathbf{e}_f \leftarrow \mathbf{e}_f + \eta\big(w(\mathbf{q}_i - \mathbf{q}_j) - \lambda \mathbf{e}_f\big), \tag{9}$$

$$\mathbf{q}_i\text{'s features } g: \ \mathbf{e}_g \leftarrow \mathbf{e}_g + \eta\big(w\,\mathbf{p}_u - \lambda \mathbf{e}_g\big), \tag{10}$$

$$\mathbf{q}_j\text{'s features } g: \ \mathbf{e}_g \leftarrow \mathbf{e}_g + \eta\big(-w\,\mathbf{p}_u - \lambda \mathbf{e}_g\big), \tag{11}$$

$$b_i\text{'s features } g: \ \beta_g \leftarrow \beta_g + \eta\big(w - \lambda \beta_g\big), \tag{12}$$

$$b_j\text{'s features } g: \ \beta_g \leftarrow \beta_g + \eta\big(-w - \lambda \beta_g\big), \tag{13}$$

with learning rate η and L2 penalty λ. This is the sense in which the model is a classical, linear SGD factorization: every update is a closed-form linear-algebra step on embeddings, with no hidden layer and no nonlinear activation anywhere in the forward pass.

A stability correction that matters in practice. A feature shared by many low-popularity training items (in our pipeline this would have been `popularity=0`, had it not been excluded from the representation per Section 4) is sampled as a WARP negative about as often as any other feature but is rarely part of a positive, so with only a weak L2 pull its bias term can diverge far faster than λ counteracts it – in early experiments this drove cold-item scores to the bottom of the ranking regardless of their brand/ingredient match. Clipping every touched embedding to a bounded L2 norm and every touched bias scalar to a bounded magnitude after each update (both configurable; LightFM's own reference implementation does the same for embeddings) removes the divergence without changing the gradient direction, only its magnitude.

5.4 Baselines

ID-only MF (BPR). The same pairwise-SGD family, but the user and item representations are raw identity rows/columns rather than feature sums – structurally identical to the sibling BPR project. A user or item absent from training has no row or column, so its score collapses to the item bias term alone (a pure popularity signal) for warm items, and to *nothing meaningful at all* for a cold item, since that item's bias was also never trained; its factor and bias are masked out of the rank for every user, warm or cold.

Most-popular. Ranks every user by the same global training-count order. Just like the ID-only baseline, a cold item has a training count of exactly zero and is therefore never recommended by this baseline either.

Random. A seeded uniform permutation per user, which calibrates the metric scale; because its coverage is total and uncorrelated with quality, high coverage alone is not evidence of a good model.

6 Evaluation Protocol

Two different notions of "holdout" are used, and keeping them distinct is the whole point of this report.

6.1 Warm-start: leave-last-one-out

Within the population of users retained after core filtering, each user's last interaction (by true review timestamp) is held out for test, the second-to-last for validation, and the rest for training. Users without enough history to leave at least three training interactions contribute only to training and are never evaluated, which avoids inflating metrics with users whose "history" is a single review.

6.2 Cold-start: entities withheld entirely

Before the warm-start split is computed, a fraction of users and a fraction of items (8% each, by default) are selected uniformly at random and every interaction touching either is removed from the training-eligible pool *before* any splitting happens. The warm-start split above is then computed only over the remainder. Three evaluation slices follow directly from this construction:

- **New user, known item** (`cold_user_eval`): reviews by a withheld user against a product that *was* trained on. Tests whether shared demographic features let the model retrieve for a user it has never seen.
- **Known user, new item** (`cold_item_eval`): reviews by a trained user against a withheld product. Tests whether shared brand/category/ingredient features let the model retrieve a product it has never seen for a user whose taste it has learned.
- **New user, new item** (`cold_both_eval`): both sides withheld – the hardest slice, reported for completeness.

Because the identity-feature vocabulary (Section 4) is built from the training partition alone, a withheld entity provably has no identity token in either the hybrid model or the ID-only baseline. The exclusion set passed to every model's ranking call is empty for these slices: there is no training history to hide, which is exactly the point.

6.3 Metrics

For every evaluated user, the model ranks the full catalogue (excluding training history, where applicable) and NDCG@K, Recall@K and MAP@K are computed against the relevant set – a single item for the warm-start partitions (leave-last-one-out), potentially several for the cold-start slices (every interaction a withheld entity has is relevant, since all of them were excluded from training).

$$\text{NDCG@}K = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u} \frac{\sum_{k=1}^{K} \mathbb{1}[\text{rel}(k)] / \log_2(k+1)}{\text{IDCG}_u}, \tag{14}$$

$$\text{Recall@}K = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u} \frac{|\{\text{relevant items retrieved in top-}K\}|}{|\text{relevant items for } u|}, \tag{15}$$

$$\text{MAP@}K = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u} \frac{1}{\min(|\text{relevant}_u|, K)} \sum_{k=1}^{K} \mathbb{1}[\text{rel}(k)] \cdot \text{Precision@}k. \tag{16}$$

6.4 Hyperparameter selection

Hyperparameters for every model (hybrid and baselines alike) are fixed before the test partition and both cold-start slices are read; the values used are recorded in `src/recommender/cli.py` (`BEST_HYBRID`, `BEST_ID_MF`) and are the same values the table-generation script (`scripts/export_results_tab` uses, so the numbers in Section 7 are reproducible from a clean checkout with `python scripts/download_data.` `&& python scripts/export_results_table.py`.

7 Results

7.1 Warm-start

On the warm-start test partition, the ID-only MF baseline leads on raw ranking quality (Table 4). This mirrors the honest negative result reported by the sibling BPR project: a model with fewer structural constraints has an easier time fitting a warm population it has already seen reviews from, because every identity it is scored on had its own row to learn. The hybrid model pays a complexity tax here – it has to route all of its signal through a comparatively small vocabulary of shared features rather than a dedicated parameter per identity – and that tax is not fully recovered by the identity feature it is also given.

7.2 Cold-start

Table 5 is the result this project exists to produce. On the *new user, known item* slice, every model (including the hybrid one) relies on an item-level bias/popularity signal, so all three non-random baselines perform comparably; the hybrid model's demographic feature sum (skin type and tone) is a weaker but genuinely non-trivial signal on its own, clearing the random floor by a wide margin.

On the *known user, new item* and *new user, new item* slices, the comparison is structural rather than a matter of degree: the popularity and ID-only baselines both report **exactly zero** NDCG, Recall and MAP. This is not a close loss – a withheld item has a training count of precisely zero, so its popularity-bias term and its identity row are both untrained, and neither baseline has any other signal to fall back on for it. The hybrid model scores strictly above zero on both slices, because its item representation routes through brand, category and ingredient-tag embeddings learned from every *other* item that shares those tags – exactly the mechanism Equation 1 was built to provide.

7.3 Honest caveats

The hybrid model's absolute recall on the cold-item slices is modest, and on this synthetic fixture's small catalogue (∼200 products) a uniform random baseline is occasionally competitive with it in raw hit-rate, simply because the ratio of K to catalogue size is already favourable to chance. We report this rather than tuning the random seed until it looks better: the qualitative claim this project supports is that the hybrid model is *capable of saying anything at all* about a withheld entity, which the two comparison baselines are structurally incapable of, not that it is dramatically better than chance on every metric at this dataset scale. A larger catalogue, more training epochs, or a sharper per-feature affinity signal would be the natural next experiments to widen the absolute margin (Section 8).

8 Conclusion

We implemented hybrid feature-aware matrix factorization with a WARP ranking loss end to end, from scratch in NumPy/SciPy: point-in-time feature engineering over demographic and catalogue attributes, a from-scratch WARP training loop with the stability corrections it needs in practice, a dedicated cold-start evaluation protocol that withholds entities from training entirely rather than merely their last interaction, and a parity-tested serving path – precomputed top-N for known entities, a documented request-time fallback for genuinely new ones – deployed by Terraform against a local AWS emulator.

The measured outcome has two parts, and we report both. On warm-start ranking quality, the simpler ID-only baseline wins, consistent with the sibling BPR project's own honest negative result: fitting a population everyone has already been seen in favours fewer structural constraints. On the cold-start slices this project was built to measure, the comparison is not close: popularity and ID-only MF are both *structurally* incapable of saying anything about a withheld entity (exactly zero on every metric), while the hybrid model retrieves a strictly positive, if modest, signal through shared brand/category/ingredient structure alone.

8.1 What transfers

- **A feature store that is the model, not an accessory to it.** Three dedicated test files (schema, leakage, parity) treat the feature layer with the same rigor the sibling projects reserve for their scoring function, because here they are the same thing.
- **A cold-start harness.** Withholding entities from training entirely, restricting identity vocabularies to the training partition, and reporting three distinct withheld-population

slices is a reusable pattern for any feature-aware recommender, not specific to skincare.

- **A from-scratch WARP implementation** with the gradient derivation and the embedding/bias clipping correction documented in Section 5, removing the fragile `lightfm` Cython dependency entirely.

8.2 Future work

Sharper negative sampling. WARP's rejection sampling degrades as training items increasingly separate into clear winners and losers; sampling negatives with a mild popularity tilt (as the sibling BPR project's own future-work notes propose) would very likely close part of the gap to the ID-only baseline on warm-start quality.

A production-scale catalogue. The honest caveat in Section 7 is partly an artifact of the synthetic fixture's small catalogue. Running the same pipeline against the real Kaggle dataset (`-kaggle`, ∼8k products and ∼1M reviews) would both sharpen the ingredient-affinity signal and lower the chance baseline's competitiveness.

Confidence intervals. Repeating the cold-start comparison across several random withheld-population draws and reporting intervals would establish how much of the zero-versus-positive gap is a reliable structural property versus a property of one particular random split.

Re-admitting popularity as a tempered prior. Section 4 excludes `item_popularity_bucket` from the retrieval representation entirely because an unweighted inclusion suppressed cold items. A middle ground – including it with a separately, more aggressively regularized embedding dimension, or only for warm items at inference time – might recover some of the warm-start ranking quality lost to the ID-only baseline without reintroducing the cold-item suppression this report documents.

References

[1] Maciej Kula. Metadata embeddings for user and item cold-start recommendations. *Proceedings of the 2nd Workshop on New Trends on Content-Based Recommender Systems (CBRecSys), CEUR Workshop Proceedings*, 1448:14–21, 2015.

[2] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 452–461, 2009.

[3] Jason Weston, Samy Bengio, and Nicolas Usunier. WSABIE: Scaling up to large vocabulary image annotation. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2764–2770, 2011.

Table 3: Per-feature rationale, generated from the `rationale` field every entry in `conf/features.yaml` is required to carry (`test_features_schema.py` rejects a feature without one).

`user_skin_type` Core demographic attribute driving taste. Known at signup, so it is available for every user including ones with zero training history – the attribute a brand-new user supplies is exactly what lets the hybrid model place them near users with similar skin.

`user_skin_tone` Secondary demographic attribute (shade matching for base/coverage products). Same availability argument as skin type.

`user_identity` Per-user ID feature, LightFM-style: lets the model retain personalised signal for warm users without it being the *only* signal. The vocabulary is restricted to users seen in training, so a cold user genuinely has no ID token available – this is the feature that operationalizes "cold start" rather than merely asserting it.

`item_brand` Brand loyalty is a strong prior independent of any single product's review history, and is known the moment a product is catalogued.

`item_category` Coarse product type (serum vs. cleanser vs. sunscreen). Known at catalogue time, safe for cold items by construction.

`item_price_bucket` Discretized price tier. Budget shoppers and prestige shoppers cluster on this signal regardless of brand or category.

`item_ingredient_tags` The content signal that actually explains taste: a Dry-skin user's affinity is for Hyaluronic-Acid / Ceramide products specifically, not for "Moisturizers" as a category. This is what lets the model recommend a brand-new product that happens to share tags with liked products.

`item_identity` Mirrors user_identity: vocabulary restricted to items seen in training, so a cold item has no ID token and must be scored through its brand/category/ingredient tokens alone.

`user_review_count_bucket` Confidence weighting: a user with 40 training reviews has a much more reliable taste signal than one with 3. Computed with a strict '<' cutoff against the user's own interaction_order so it can never see the very review it is being used to help predict.

`item_popularity_bucket` Popularity prior used by the most-popular baseline and as part of the cold-start fallback ordering. Fitted on the training partition only, so held-out interactions never inflate a product's apparent popularity.

`item_catalogue_context` Distinguishes an "established" item (enough training reviews to trust its popularity/ingredient signal) from a "new_arrival" (sparse or zero training reviews). Cold items always fall back to this value, documenting that they are indistinguishable from genuinely new catalog entries from the model's point of view.

Table 4: Warm-start ranking quality on the held-out test partition ($K = 10$). Hyperparameters were fixed before the test partition was read once.

Model	NDCG@10	Recall@10	MAP@10	Hit rate	Coverage
ID-only MF	**0.1120**	0.2084	0.0827	0.2084	0.8950
Popularity	0.0788	0.1508	0.0572	0.1508	0.1000
Hybrid WARP	0.0655	0.1286	0.0466	0.1286	0.8900
Random	0.0364	0.0687	0.0265	0.0687	1.0000

Table 5: Cold-start retrieval quality ($K = 10$). Users/items in this slice were withheld from training entirely; the ID-only baseline and the popularity baseline can only fall back to a popularity prior for them, while the hybrid model scores them through shared category, brand and ingredient features.

Model	Slice	NDCG@10	Recall@10	MAP@10	Hit rate
Hybrid WARP	New user, new item	0.0282	0.0758	0.0126	0.0909
Random	New user, new item	0.0236	0.0389	0.0098	0.1212
Popularity	New user, new item	0.0000	0.0000	0.0000	0.0000
ID-only MF	New user, new item	0.0000	0.0000	0.0000	0.0000
Random	Known user, new item	0.0308	0.0570	0.0160	0.1084
Hybrid WARP	Known user, new item	0.0129	0.0328	0.0047	0.0623
Popularity	Known user, new item	0.0000	0.0000	0.0000	0.0000
ID-only MF	Known user, new item	0.0000	0.0000	0.0000	0.0000
Popularity	New user, known item	0.2710	0.1378	0.1333	0.9250
ID-only MF	New user, known item	0.2661	0.1268	0.1363	0.8500
Hybrid WARP	New user, known item	0.1956	0.0970	0.0888	0.7750
Random	New user, known item	0.0642	0.0438	0.0197	0.5250